

Generación de Música en ABC Notation con Redes Recurrentes LSTM

Kevin Cárdenas

Universidad San Francisco de Quito / Aprendizaje Profundo

kcardenasl@estud.usfq.edu.ec

Resumen

Este trabajo presenta un modelo de red neuronal recurrente (LSTM) para generar música en notación ABC a nivel de carácter. El modelo se entrenó con un corpus de 4,094 piezas de música tradicional irlandesa (1,68M caracteres), usando una ventana de contexto de 100 caracteres con traslape exhaustivo. Se compararon dos regímenes de entrenamiento: el primero fijó 20 épocas y terminó con una pérdida de validación de 0,4637; el segundo empleó *early stopping*, que se detuvo en la época 46 tras cinco épocas sin mejora y alcanzó una pérdida de validación de 0,4389. El texto generado se validó sintácticamente con `music21` y se convirtió a audio mediante una cadena ABC→MIDI→WAV. Esto muestra que el enfoque puede producir composiciones nuevas y reproducibles.

1. Introducción

La generación automática de música ha sido abordada desde múltiples representaciones: audio crudo, rollos de piano (piano roll), MIDI simbólico y notación textual. Este proyecto usa **ABC notation**, un formato de texto plano muy usado para transcribir música folk, el cual codifica notas, duraciones y metadatos (título, métrica, tonalidad) como caracteres ASCII. Esta representación permite tratar la generación musical como un problema de *modelado de lenguaje a nivel de carácter*, análogo a la generación de texto natural.

Las redes neuronales recurrentes y en particular las Long Short-Term Memory (LSTM), son la arquitectura estándar para este tipo de tareas secuenciales, ya que permiten retener dependencias de largo plazo sin sufrir el problema del gradiente que se desvanece [3]. El enfoque de generación de texto carácter-a-carácter con RNNs fue popularizado por Karpathy [4], quien demostró que este tipo de modelos puede aprender estructuras sintácticas complejas (código fuente, prosa, partituras) únicamente a partir de secuencias de caracteres. El curso MIT 6.S191 [1] aplica directamente esta idea a la generación de música en notación ABC, sirviendo como referencia metodológica directa para este proyecto.

El objetivo de este trabajo es entrenar un modelo LSTM capaz de generar secuencias nuevas y sintácticamente válidas en notación ABC, a partir de un corpus de música tradicional irlandesa [5], y convertir dicha salida en audio reproducible como validación cualitativa adicional del resultado.

2. Metodología

2.1. Dataset

Se utilizó el conjunto de datos *abc-notation-of-tunes* [5], disponible en Kaggle, el cual corresponde a una transcripción de la colección de O'Neill's de música tradicional irlandesa. El corpus contiene 4,094 piezas (*tunes*) concatenadas en un único archivo de texto, totalizando 1,684,914 caracteres. El vocabulario a nivel de carácter resultante consta de 95 sím-

bolos únicos, que incluyen notas musicales, indicadores de duración, decoraciones (ligaduras, adornos), y los campos de encabezado propios de la notación ABC (X número de referencia, T título, M métrica, L duración por defecto, K tonalidad, R tipo de ritmo, B y N notas bibliográficas, y Z transcriptor).

2.2. Preprocesamiento

El texto completo fue codificado como una secuencia de enteros mediante un mapeo carácter-a-índice. Para el entrenamiento se definieron ventanas de contexto de tamaño fijo (CONTEXT_WINDOW=100), generadas con un desplazamiento de un carácter (STRIDE=1). Esto produce un traslape exhaustivo entre ventanas consecutivas: cada ventana comparte 99 de sus 100 caracteres con la anterior, desplazándose solo una posición a la vez en vez de particionar el texto en bloques disjuntos. Esta técnica maximiza la cantidad de secuencias de entrenamiento disponibles a partir de un corpus de tamaño fijo, siguiendo el enfoque estándar para modelos de lenguaje a nivel de carácter.

El corpus fue dividido en 90 % para entrenamiento y 10 % para validación, resultando en 1,516,321 y 168,391 secuencias respectivamente. A diferencia de la división 80/20 habitual en tareas de clasificación con conjuntos de datos pequeños, se optó por un 10 % de validación dado el tamaño considerable del corpus: esta fracción ya produce un conjunto de validación robusto (168 mil secuencias), por lo que se prefirió maximizar los datos disponibles para entrenamiento en lugar de reservar una porción mayor para validación.

2.3. Arquitectura

La arquitectura del modelo está compuesta por una capa de *embedding* de 128 dimensiones, seguida de dos capas LSTM apiladas con 256 unidades ocultas y un *dropout* de 0.2. La salida de la última capa recurrente se conecta a una capa lineal que proyecta las activaciones al tamaño del vocabulario para predecir la probabilidad del siguiente carácter. La Figura 1 muestra un esquema general de esta arquitectura.

Se eligió una arquitectura LSTM [3] porque este tipo de redes es capaz de aprender dependencias de largo plazo gracias a su mecanismo de compuertas de entrada, olvido y salida, lo que ayuda a reducir el problema del desvanecimiento del gradiente presente en las RNN tradicionales.

Los hiperparámetros utilizados se seleccionaron considerando el tamaño del vocabulario y la complejidad del problema. Una dimensión de *embedding* de 128 y un estado oculto de 256 proporcionan una capacidad de representación suficiente sin aumentar innecesariamente el número de parámetros del modelo. Además, el uso de dos capas LSTM permite aprender tanto relaciones locales entre caracteres como patrones musicales de mayor alcance presentes en las secuencias de notación ABC.

Por último, se incorporó un *dropout* de 0.2 como técnica de regularización para reducir el riesgo de sobreajuste. Durante el entrenamiento, esta técnica desactiva aleatoriamente una parte de las neuronas, favoreciendo que el modelo aprenda representaciones más robustas y generalice mejor sobre datos no vistos.

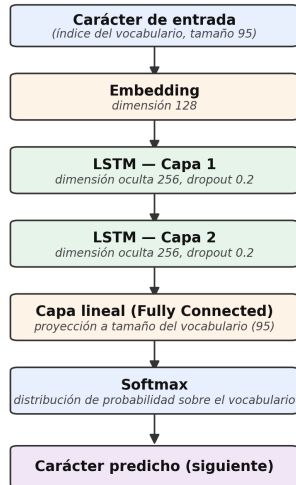


Figura 1: Arquitectura del modelo LSTM utilizado.

2.4. Entrenamiento

El modelo fue optimizado con Adam (*learning rate* 3×10^{-3}) minimizando la entropía cruzada entre el carácter predicho y el real, procesando lotes (*batches*) de 128 secuencias en paralelo por cada paso de optimización. Se evaluaron dos configuraciones:

- **Entrenamiento principal:** 20 épocas fijas, sin criterio de parada temprana.
- **Entrenamiento extendido:** con *early stopping* (paciencia de 5 épocas sin mejora en la pérdida de validación, techo máximo de 50 épocas).

En ambos casos se implementó *checkpointing*: el estado completo del entrenamiento se guarda en cada época (permi-

tiendo reanudar tras interrupciones), y los pesos correspondientes a la menor pérdida de validación se conservan por separado como el modelo final para generación.

3. Resultados y Discusión

3.1. Curvas de entrenamiento

La Figura 2 muestra la evolución de la pérdida durante el entrenamiento principal de 20 épocas. La pérdida de entrenamiento disminuyó de 0.6755 a 0.4365, mientras que la de validación pasó de 0.6274 a un mínimo de 0.4710 en la época 18, sin señales de sobreajuste severo, aunque la curva de validación comienza a aplanarse hacia el final.

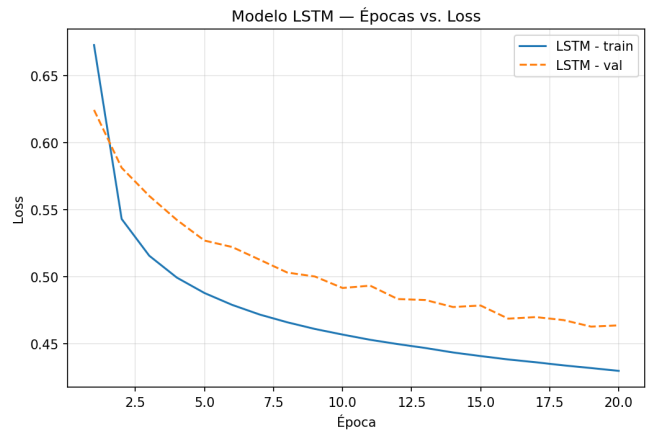


Figura 2: Épocas vs. pérdida, entrenamiento principal (20 épocas).

La Figura 3 muestra la misma evolución para el entrenamiento extendido con *early stopping*, que se detalla a continuación.

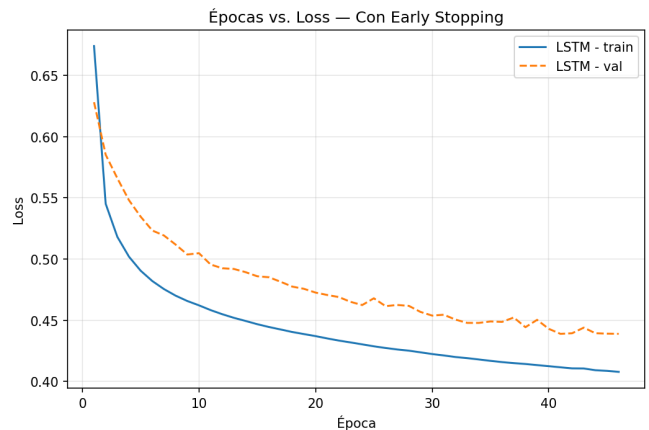


Figura 3: Épocas vs. pérdida, entrenamiento con *early stopping*.

3.2. Entrenamiento extendido con early stopping

Para verificar si el modelo de 20 épocas había convergido, se extendió el entrenamiento con *early stopping* (paciencia de 5 épocas, techo máximo de 50). El criterio de parada se activó en la época 46, al no observarse mejora en la pérdida de validación durante 5 épocas consecutivas. El mejor modelo se obtuvo en la época 41, con una pérdida de validación de 0.4389 (perplejidad de 1.55), una mejora notable respecto al 0.4637 (perplejidad de 1.60) obtenido al finalizar las 20 épocas fijas. Este resultado confirma que el punto de convergencia real se ubica considerablemente más allá de ese rango, aunque dentro de un horizonte moderado (menos de 50 épocas), sin señales de inestabilidad ni necesidad de un entrenamiento excesivamente prolongado.

3.3. Generación y validez sintáctica

Para generar música, se le da al modelo un pequeño inicio de texto ("X:1\ nT:", el comienzo típico de una pieza en ABC) y a partir de ahí escribe carácter por carácter, con algo de aleatoriedad ($T = 0,8$) para que cada resultado sea distinto. El modelo completa correctamente el encabezado (número, título, métrica, tonalidad, etc.) e inventa títulos que no existen en el corpus original, lo que muestra que aprendió el patrón general y no solo memorizó piezas del dataset. Al validar las muestras generadas con *music21* [2], se confirmó que son ABC sintácticamente válido. Sin embargo, en algunos casos *abc2midi* mostró advertencias de compases con más notas de las que la métrica permite (por ejemplo, 7 unidades de tiempo en un compás de 6/8), lo que indica que el modelo aprende bien la estructura general, pero no siempre respeta con exactitud las reglas de duración dentro de cada compás.

Este tipo de inconsistencia es comparable a la reportada por Sturm et al. [6], quienes entrenaron una LSTM carácter-a-carácter sobre un corpus de transcripciones ABC de música celta considerablemente mayor (23,000 piezas, vocabulario de 135 caracteres, tres capas de 512 unidades). En su análisis estadístico, encontraron que una fracción de las transcripciones generadas presentaba signos de repetición mal emparejados (por ejemplo, el token de primera terminación |1 seguido de otro |1 en vez de |2) y acordes incompletos, errores estructurales puntuales, análogos en naturaleza al descuadre métrico observado en este trabajo, pese a la diferencia sustancial de escala entre ambos modelos. Vale la pena señalar, además, que los propios autores reportan que su corpus de entrenamiento ya contenía miles de compases con conteo incorrecto de notas antes incluso de entrenar el modelo, y que no intentaron corregir dichos problemas, lo que sugiere que este tipo de inconsistencia estructural puede heredarse directamente de la calidad del corpus original, no ser exclusivamente una limitación del modelo en sí. Asimismo, dicho trabajo utiliza una división 95 %/5 % para entrenamiento y validación, lo que respalda el criterio usado en este proyecto de reservar una fracción reducida de validación dado el tamaño del cor-

pus.

3.4. Conversión a audio

El texto ABC generado fue convertido a MIDI mediante *abc2midi* y sintetizado a audio real (WAV) mediante *FluidSynth* con el banco de sonidos *FluidR3_GM*, permitiendo una validación cualitativa adicional del resultado mediante escucha directa. La duración de las piezas generadas varía considerablemente (desde 10 segundos hasta más de un minuto) dependiendo de la longitud del encabezado generado y de la presencia de signos de repetición, que *abc2midi* expande literalmente al render de audio.

4. Conclusiones

Este trabajo demostró que una red LSTM entrenada a nivel de carácter es capaz de aprender la estructura sintáctica de la notación ABC, incluyendo encabezados, métrica, tonalidad y patrones melódicos característicos de la música tradicional irlandesa, y generar piezas nuevas, sintácticamente válidas y no presentes en el corpus original. La conversión completa del texto generado a audio real (ABC→MIDI→WAV) confirma que el enfoque es viable de extremo a extremo, no solo a nivel textual.

La comparación entre el entrenamiento de 20 épocas fijas y el entrenamiento extendido con *early stopping* reveló que el modelo principal se encontraba subentrenado: el criterio de parada temprana se activó en la época 46, alcanzando una pérdida de validación de 0.4389, notablemente menor que el 0.4637 obtenido en las 20 épocas iniciales. Esto confirma que, con un número moderado de épocas adicionales, el modelo tiene margen de mejora antes de estabilizarse, sin requerir un entrenamiento excesivamente extenso.

Como limitaciones del trabajo se identifican: (i) el modelo no garantiza consistencia métrica estricta dentro de cada compás, como evidenciaron las advertencias de *abc2midi* en algunas muestras generadas; (ii) la validación sintáctica se realizó de forma cualitativa sobre muestras individuales, sin una métrica agregada sobre un número mayor de generaciones; y (iii) la duración de las piezas generadas varía considerablemente debido a la longitud variable del encabezado y a la expansión de signos de repetición durante la conversión a audio.

Como trabajo futuro se propone: calcular una métrica cuantitativa de validez sintáctica sobre un conjunto mayor de muestras generadas, y explorar ajustes de temperatura de muestreo para balancear diversidad y coherencia estructural en las piezas generadas.

Referencias

- [1] Alexander Amini, Ava Soleimany, et al. Mit 6.s191: Introduction to deep learning. <https://introtodeeplearning.com>, 2022.

- [2] Michael Scott Cuthbert and Christopher Ariza. music21: A toolkit for computer-aided musicology and symbolic music data. In *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, 2010.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] Andrej Karpathy. The unreasonable effectiveness of recurrent neural networks. <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>, 2015.
- [5] Francis O’Neill. O’neill’s music of ireland. Colección transcrita a notación ABC, disponible en Kaggle, 1903.
- [6] Bob L. Sturm, João Felipe Santos, Oded Ben-Tal, and Iryna Korshunova. Music transcription modelling and composition using deep learning. In *Proceedings of the 1st Conference on Computer Simulation of Musical Creativity*, Huddersfield, UK, 2016.